

Cyber Security Internship – Task 2

OWASP Top 10 Vulnerability Analysis

Jobin sabu
VedGrow Internship Programme – CS Track

July 2026

0.1 Scope & Disclaimer

All demonstrations in this report were performed against **DVWA (Damn Vulnerable Web Application)**, running locally in a Docker container on my own machine (`localhost:8080`), with Security Level set to **Low**. No external or third-party websites were tested. This complies with the authorised-use policy of the VedGrow internship programme.

- **Target application:** DVWA v1.10, local Docker instance
- **Testing account:** admin (default DVWA credentials)
- **Categories demonstrated live:** A03 Injection (SQL Injection, Blind SQL Injection, Reflected XSS, Stored XSS, Command Injection)
- **Remaining categories:** explained conceptually below, with vulnerable vs. secure code, since DVWA does not provide a dedicated lab for every OWASP category

Part A: Hands-On Demonstrations

0.2 Demo 1: SQL Injection

OWASP Category: A03:2021 – Injection

Payload used:

```
1' OR '1'='1
```

Result: The application's WHERE clause was manipulated into a condition that is always true, causing the database to return **every user record** instead of the single record the form expects.

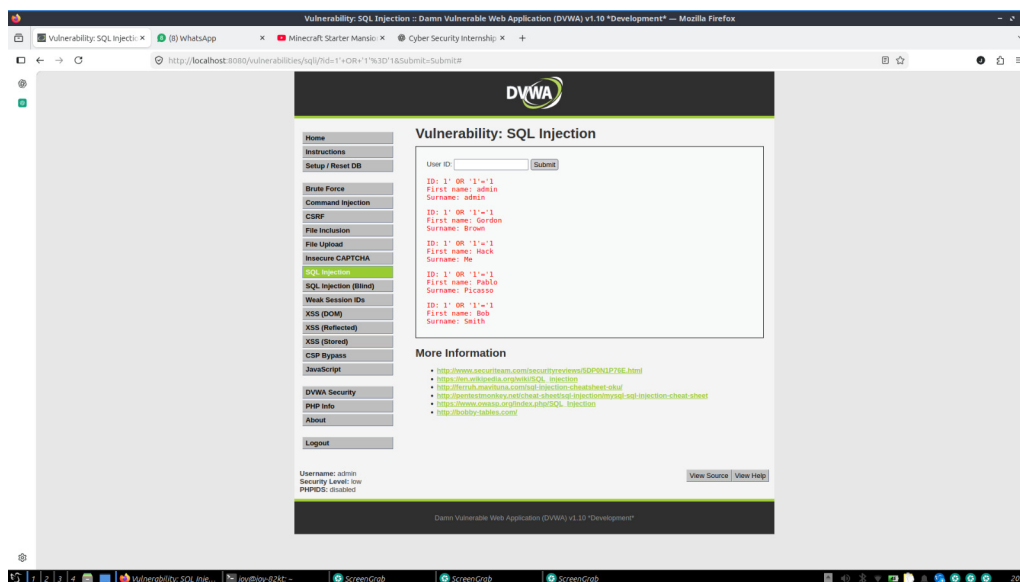


Figure 1: SQL Injection – all user records returned via 1' OR '1'='1

Mitigation: Use parameterised queries / prepared statements; never concatenate user input directly into SQL strings; apply least privilege to the database account.

0.3 Demo 2: Blind SQL Injection

OWASP Category: A03:2021 – Injection

Unlike regular SQL Injection, Blind SQLi does not return visible data – the attacker infers true/false information purely from how the application's response text changes.

Payload 1 (true condition):

```
1' AND '1'='1
```

Result: *"User ID exists in the database."*

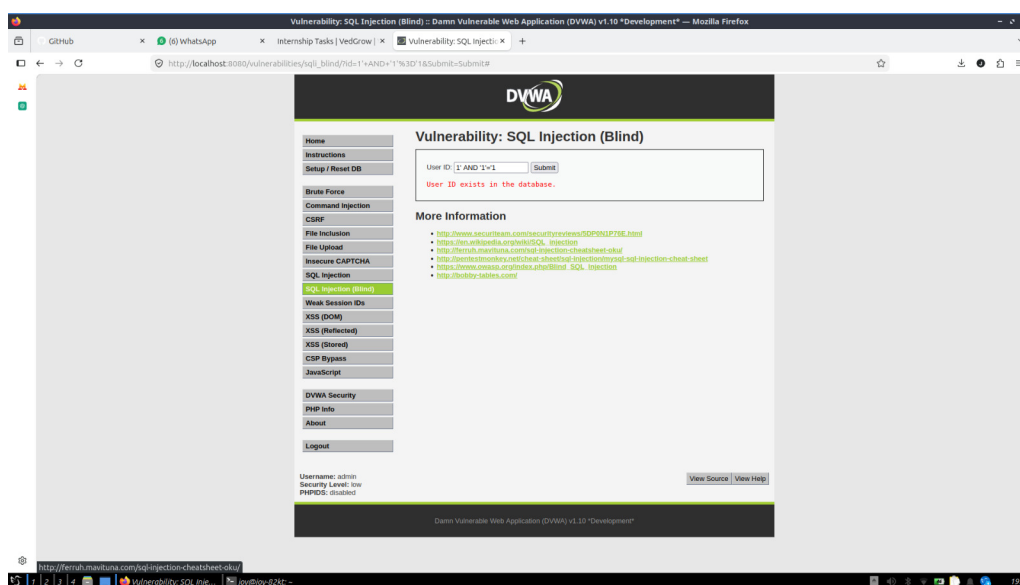


Figure 2: Blind SQLi – true condition confirms the User ID exists

Payload 2 (false condition):

```
1' AND '1'='2
```

Result: *"User ID is MISSING from the database."*

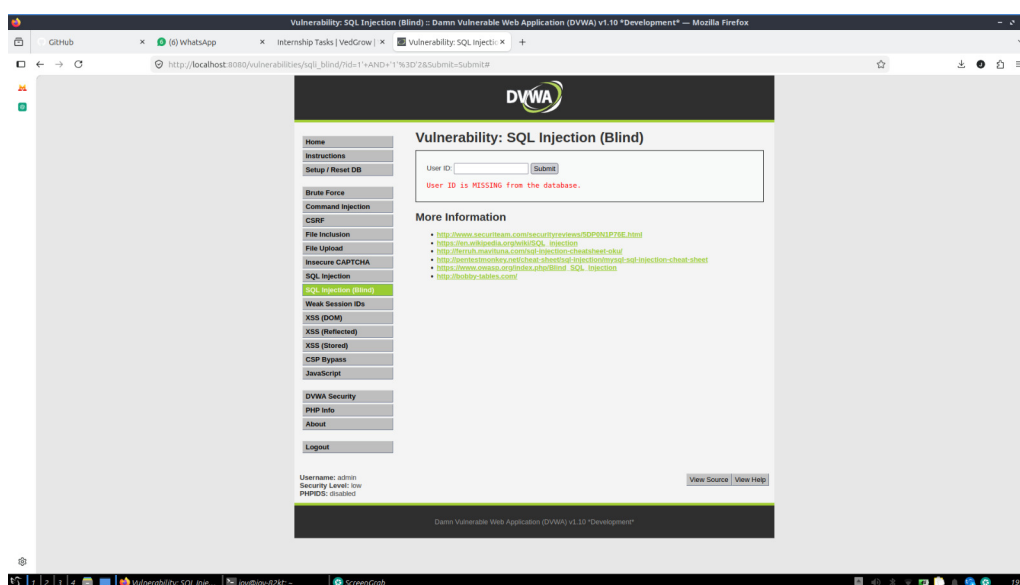


Figure 3: Blind SQLi – false condition returns a different response

Analysis: The contrast between the two responses proves the input is being interpreted as SQL rather than as literal data. In a real attack, this true/false oracle can be used to extract data one character at a time (e.g. testing `SUBSTRING(password,1,1)='a'`), which is slower than regular SQLi but equally dangerous.

Mitigation: Identical to standard SQL Injection – parameterised queries eliminate both variants at once.

0.4 Demo 3: Reflected Cross-Site Scripting (XSS)

OWASP Category: A03:2021 – Injection (XSS)

Payload used (in URL parameter):

```
<script>alert('XSS')</script>
```

Result: The application echoed the parameter directly into the page's HTML without encoding it, so the browser executed it as real JavaScript rather than displaying it as text – producing an alert popup.

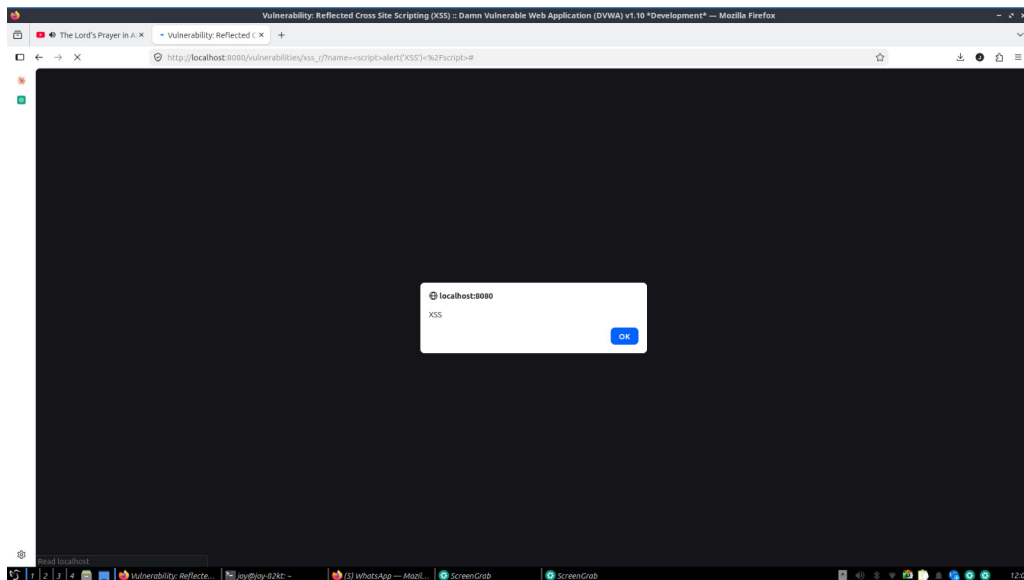


Figure 4: Reflected XSS – script executes from a crafted URL

Why “reflected”: The payload exists only in the URL and is not stored – it only fires when a victim is tricked into clicking a maliciously crafted link.

Mitigation: HTML-escape all user input before rendering it into the page (e.g. `html.escape()` in Python, or template auto-escaping); apply a Content-Security-Policy header.

0.5 Demo 4: Stored Cross-Site Scripting (XSS)

OWASP Category: A03:2021 – Injection (XSS)

Payload used (submitted via guestbook form):

Name: Bob

Message: `<script>alert('Stored XSS')</script>`

Result: The payload was saved to the database. On reloading the page – without resubmitting anything – the alert fired again, proving the script now executes automatically for **any** visitor to the page, not just the original submitter.

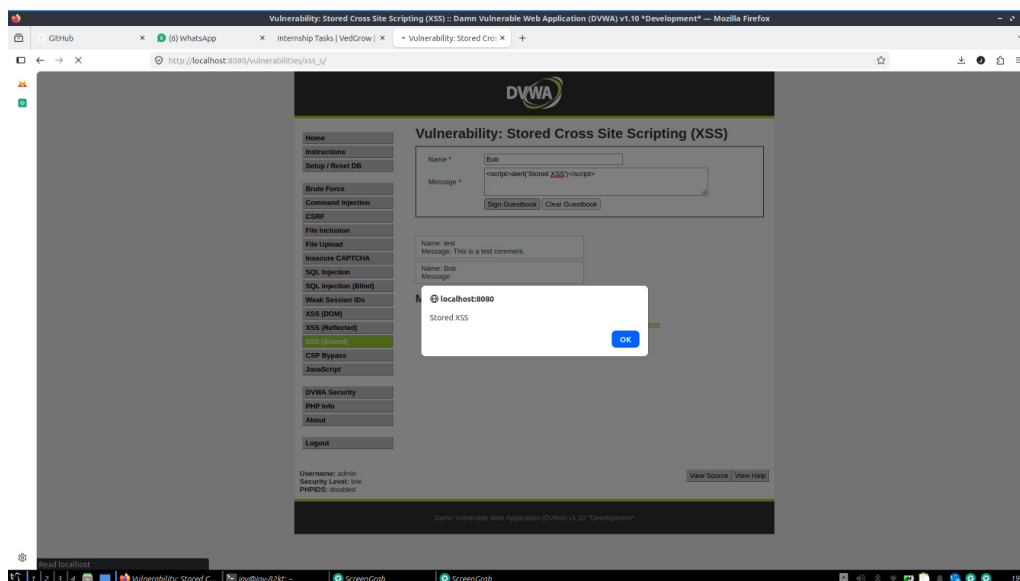


Figure 5: Stored XSS – payload persists server-side and executes on page load

Why this is more severe than Reflected XSS: No crafted link or social engineering is required – every user who simply views the page is affected.

Mitigation: Same as Reflected XSS (output encoding, CSP) applied consistently to *all* stored content, plus input validation at the point of submission.

0.6 Demo 5: Command Injection

OWASP Category: A03:2021 – Injection

Payload used (in the “ping a device” field):

```
127.0.0.1; whoami
```

Result: The application passed the input directly to the system shell. Below the normal ping output, the injected `whoami` command executed and returned `www-data` – proof of arbitrary OS command execution.

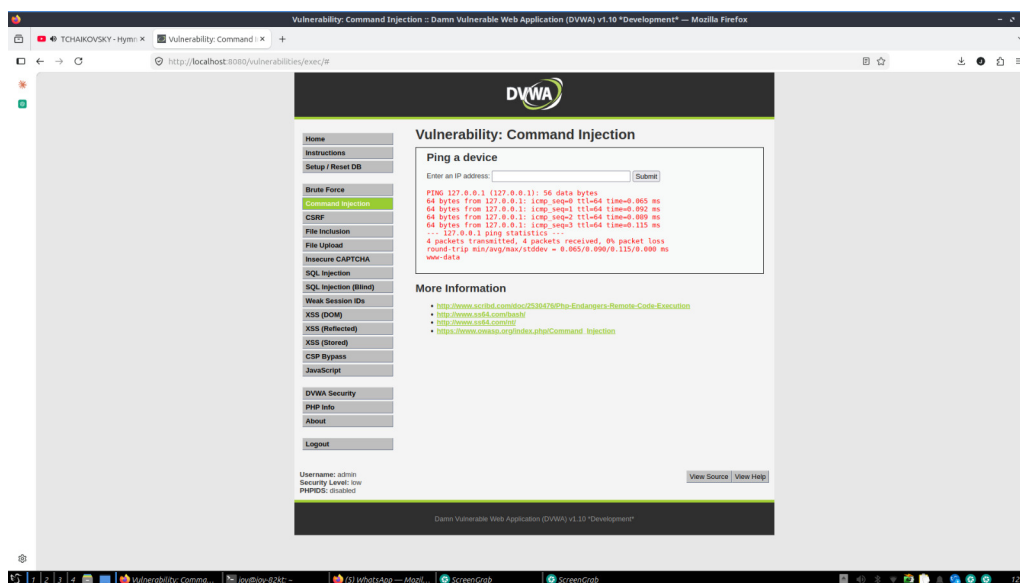


Figure 6: Command Injection – `whoami` output appended after ping results

Mitigation: Never build shell commands from user input; use library calls with argument arrays instead of a shell (e.g. `subprocess.run(..., shell=False)`); validate input against a strict allowlist (e.g. valid IP address format only).

Part B: Full OWASP Top 10 Analysis

The following covers all ten OWASP categories. Five were demonstrated hands-on above (all falling under A03: Injection, in its various forms). The remainder are explained here with vulnerable vs. secure code, since DVWA does not provide a dedicated lab for every category.

0.7 Broken Access Control

Plain English: The application checks *who* you are but not *what* you're allowed to do, letting users access data or functions above their privilege level.

Vulnerable code:

```
@app.route("/invoice/<invoice_id>")
def view_invoice(invoice_id):
    invoice = db.get_invoice(invoice_id) # no ownership check
    return render_template("invoice.html", invoice=invoice)
```

Secure code:

```
@app.route("/invoice/<invoice_id>")
@login_required
def view_invoice(invoice_id):
    invoice = db.get_invoice(invoice_id)
    if invoice is None or invoice.owner_id != current_user.id:
        abort(403)
    return render_template("invoice.html", invoice=invoice)
```

Prevention: Deny by default; enforce authorisation server-side on every request; log access-control failures.

0.8 Cryptographic Failures

Plain English: Sensitive data is exposed due to missing or weak encryption – either in transit (no TLS) or at rest (weak hashing).

Vulnerable code:

```
import hashlib
def store_password(password):
    return hashlib.md5(password.encode()).hexdigest() # broken, unsalted
```

Secure code:

```
import bcrypt
def store_password(password):
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))
```

Prevention: TLS everywhere; bcrypt/Argon2id for passwords; never MD5/SHA-1; AES-256-GCM for data at rest.

0.9 Injection

Plain English: User input is passed into an interpreter (SQL, shell, LDAP) as executable code instead of inert data.

Demonstrated live above: SQL Injection, Blind SQL Injection, Reflected XSS, Stored XSS, Command Injection.

Vulnerable code:

```
query = "SELECT * FROM users WHERE id = " + user_id # string concat
```

Secure code:

```
query = "SELECT * FROM users WHERE id = ?"
db.execute(query, (user_id,)) # parameterised
```

Prevention: Parameterised queries; ORMs; output encoding for XSS; avoid shell execution entirely for command-style features.

0.10 Insecure Design

Plain English: A missing security requirement baked in from the start – not a single coding bug, but an architectural gap (e.g. no threat modelling for abuse cases).

Vulnerable pattern:

```
price = request.form.get("price") # trusts client-supplied price
charge_card(current_user, price)
```

Secure pattern:

```
item = db.get_item(request.form.get("item_id"))
charge_card(current_user, item.price) # server is source of truth
```

Prevention: Threat model early (STRIDE); write abuse-case user stories; never trust client-modifiable data.

Note: DVWA has no dedicated module for this category – WebGoat’s “Insecure Design” lesson provides a live example.

0.11 Security Misconfiguration

Plain English: Insecure defaults left in place – default credentials, verbose error messages, unnecessary features enabled.

Live evidence: DVWA itself demonstrates this category – default credentials (admin/password) and the Security Level setting (Low/Medium/High/Impossible) directly control how much misconfiguration is present.

Vulnerable code:

```
app.config["DEBUG"] = True # exposes stack traces in production
```

Secure code:

```
app.config["DEBUG"] = False
@app.errorhandler(500)
def internal_error(e):
    app.logger.error(e)
    return render_template("500.html"), 500
```

Prevention: Change all default credentials; disable debug mode in production; remove unused features/ports.

0.12 Vulnerable and Outdated Components

Plain English: The application relies on libraries or platforms with known, publicly disclosed CVEs that were never patched.

Detection commands:

```
pip-audit          # Python dependencies
npm audit          # Node dependencies
nmap -sV -p- 127.0.0.1 # fingerprint own running services
```

Prevention: Maintain a dependency inventory; automate scanning in CI/CD; patch on a severity-based SLA.

Note: DVWA has no dedicated module – WebGoat’s “Vulnerable Components” lesson demonstrates this directly.

0.13 Identification and Authentication Failures

Plain English: Weaknesses in login or session handling – no rate-limiting, weak password checks, predictable sessions – let an attacker impersonate a legitimate user.

Vulnerable code:

```
if user and user.password == request.form["password"]: # no logout
    session["user_id"] = user.id
```

Secure code:

```
@limiter.limit("5_per_minute")
def login():
    if user and bcrypt.checkpw(request.form["password"].encode(),
                               user.pw_hash):
        session.clear()
        session["user_id"] = user.id
```

Prevention: MFA; rate-limit logins; bcrypt for password checks; rotate session IDs on login.

DVWA’s Brute Force module is directly relevant here and can be added as a sixth demo if desired.

0.14 Software and Data Integrity Failures

Plain English: The application trusts data or code without verifying it hasn’t been tampered with – e.g. deserialising untrusted data into live objects.

Vulnerable code:

```
import pickle
def load_session(data):
    return pickle.loads(data) # untrusted deserialisation = code exec risk
```

Secure code:

```
import json
def load_session(data):
    return json.loads(data) # no executable payload capability
```

Prevention: Never deserialise untrusted data with formats that support executable payloads; verify checksums/signatures on artifacts.

Note: WebGoat’s “Insecure Deserialization” lesson demonstrates this exploit chain end-to-end.

0.15 Security Logging and Monitoring Failures

Plain English: Attacks go unnoticed because security-relevant events aren't logged or monitored. This category is about an *absence* of controls rather than a single exploitable endpoint, so neither DVWA nor WebGoat provides a dedicated lab.

Vulnerable code:

```
if not valid:
    return "Invalid_credentials", 401    # silent failure, nothing recorded
```

Secure code:

```
if not valid:
    security_log.warning("failed_login_user=%s_ip=%s",
                        request.form.get("username"), request.remote_addr)
    return "Invalid_credentials", 401
```

Prevention: Log every auth event and access-control failure with context; centralise logs (SIEM); alert on anomalies.

0.16 Server-Side Request Forgery (SSRF)

Plain English: The server fetches a user-supplied URL without restriction, allowing an attacker to reach internal-only services (cloud metadata endpoints, internal databases) that were never meant to be internet-facing.

Vulnerable code:

```
url = request.args.get("url")
resp = requests.get(url)    # fetches ANY URL, including internal ones
```

Secure code:

```
ALLOWED_HOSTS = {"images.trustedcdn.com"}
host = urlparse(url).hostname
if host not in ALLOWED_HOSTS:
    abort(400)
```

Prevention: Allow-list permitted destination hosts; block requests to private/loopback IP ranges; disable dangerous URL schemes.

Note: WebGoat's "Server-Side Request Forgery" lesson provides a purpose-built vulnerable endpoint.